Contents

1 Abstract **2**

2 Introduction **2**

3 Ray Tracing **3**

3.1 Vectors and Coordinates 3

1 Abstract

Add abstract

2 Introduction

When you look at the world through your eyes, you are seeing a 3-dimensional space. In contrast, the image that you see possesses only 2 dimensions, and yet, we recognize our view as a 3D one. Through this view, one can still perceive the depth of such an image, retrieving some of the information lost in the process of flattening the view into 2 dimensions. This phenomenon reveals the possibility of projecting a 3D space onto a 2D plane, allowing us to "emulate" the existence of a 3D space behind a 2D plane (much like looking through a window to look at the outside world; a detailed enough 2D image on the window would be indistinguishable from the outside view).

In truth, this possibility is taken advantage of to form the foundations of modern computer graphics. When it comes to video games, computer generated imagery, or 3D animation, computers must simulate all the physical processes of the real world that produce the light and resulting image that reaches your eyes. Although the entirety of the physical world cannot possibly be simulated within the constraints of modern computing capabilities, mathematical and physical abstractions can be made in order to simplify this process and produce near true to life results. These abstractions form the basis of 3D Computer Graphics.

In modern computing, knowing the fundamental concepts and processes behind projecting a 3D image has been made unnecessary to know. With the release of many software solutions and libraries that aid in the development of appropriate visuals, the modern day developer does not need to grasp the underlying mechanics behind 3D Computer Graphics in order to produce satisfactory results.

However, the benefits of learning the fundamental process behind this widely used field are essential for those who wish to become true experts not tied down by any specific engine or library. Factors such as performance, scalability, and customizability are reasons that justify the process of learning the fundamentals. Above all else, those who understand are better equipped to take advantage of and develop graphical toolkits. As such, it is pertinent to understand the foundations that make up 3D Computer Graphics for those who wish to become true masters of the field.

Add real-world examples of mentioned advantages

This research paper sets out to document the process of learning the fundamentals of 3D Computer Graphics, with real code examples mapping out the learning process. As such, it aims to serve as a useful guide that directs readers to explore the bare-bones fundamentals of 3D Computer Graphics.

Add limitations of this paper

3 Ray Tracing

3.1 Vectors and Coordinates

In order to represent a 3-dimensional space, we must first develop the tools used for defining a 3D space.

A **cartesian coordinate system** is a system of numerically representing space with a set of coordinates. n-dimension spaces require n sets of coordinates to represent. As such, a 3-dimension cartesian coordinate system requires 3 sets of coordinates, commonly referred to as x, y, and z.

A **point** is a single position in space, represented by a set of coordinates, and notated by $(C_1, C_2, C_3, \ldots, C_n)$. A 3D cartesian coordinate system places the center, or the origin point at $(0, 0, 0)$. Each dimension, referred to as an axis, has a direction: traveling towards the direction of an axis is known as traveling positively, while traveling against the direction of an axis is known as traveling negatively, with the corresponding coordinates increasing or decreasing appropriately.

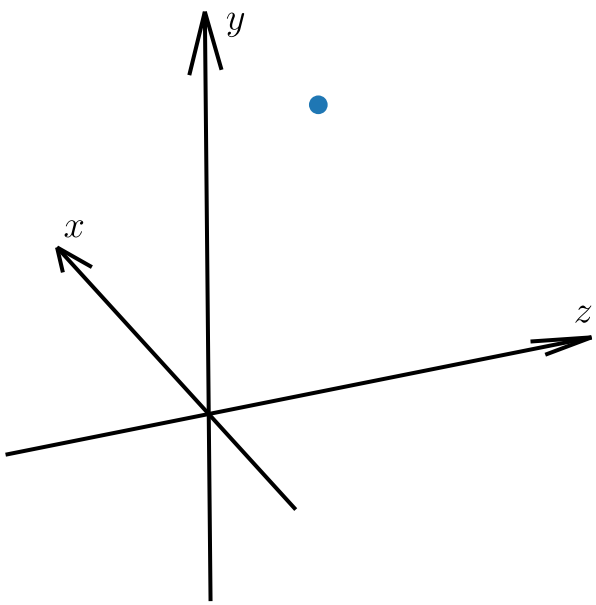


Figure 1: The point (1, 1, 1) in a 3D cartesian coordinate system.

The directions of each axis in relation to each other can be remembered using the right hand rule.

Axis configurations corresponding to the left hand also exist, however the right-handed system is more common.

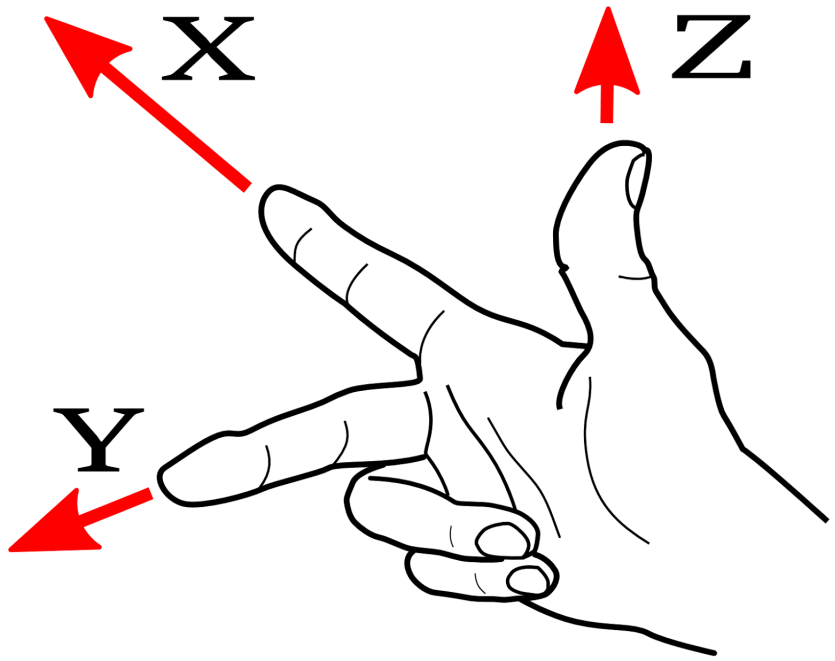


Figure 2: Right-hand rule

Vectors are the differences between points. Vectors can be used to represent many things, such as motion and direction. At its simplest, it can be intuitively thought of as a set of components that represent a certain magnitude of motion in a certain direction. It can also be thought of as the concept of a slope used in 2D cartesian coordinate systems expanded for n-dimensions. Visually, it is a line connecting two points in space (Figure 3).

n-dimension vectors can be notated as a set of n components $(C_1, C_2, C_3, \ldots, C_n)$. Much like the slope of a line in 2 dimensions, vectors can be thought of as the ratio between each of the components' deltas (differences in magnitude). For example, take the 3D vector $(1, 2, 3)$: when x is increased by 1, y is increased by 2, and z is increased by 3 (Figure 4).

Now that we have established a system of rules for representing space numerically, we can begin to lay the groundwork for projecting a 3D space onto a 2D one.

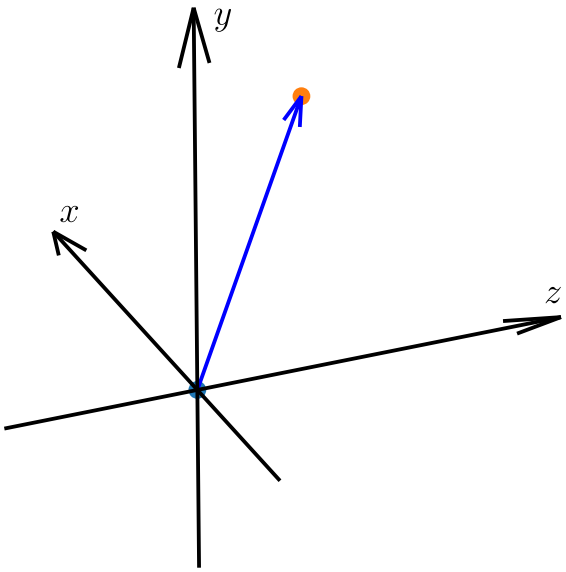


Figure 3: A vector from points $(0, 0, 0)$ to $(1, 1, 1)$

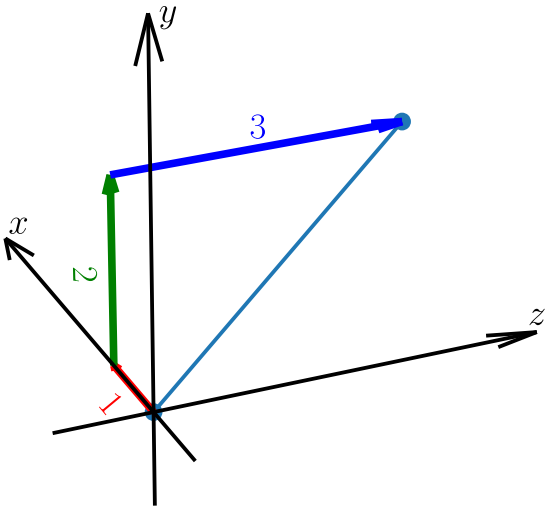


Figure 4: The vector $(1, 2, 3)$ broken down into its x, y, and z components